Flight Booking System

1. Domain Overview

Search → Select Flight → Validate Price → Create Booking → Pay → Confirm Ticket → View/Cancel Booking

---

2. *End-to-End System Workflow*

Search Flights → Display Available Flights → View Flight Details → Select Flight → Validate Availability and Price → Create Booking with PENDING_PAYMENT Status → Add Passenger Details → Select Seats → Create Payment Request → Process Payment through Payment Gateway → Receive Payment Result → Update Payment Status → Confirm Booking after Successful Payment → Generate Ticket → Send Booking Confirmation Email → Send Ticket Email → View Booking / View Ticket → Cancel Booking if Eligible → Process Refund if Required → Send Cancellation / Refund Email

---

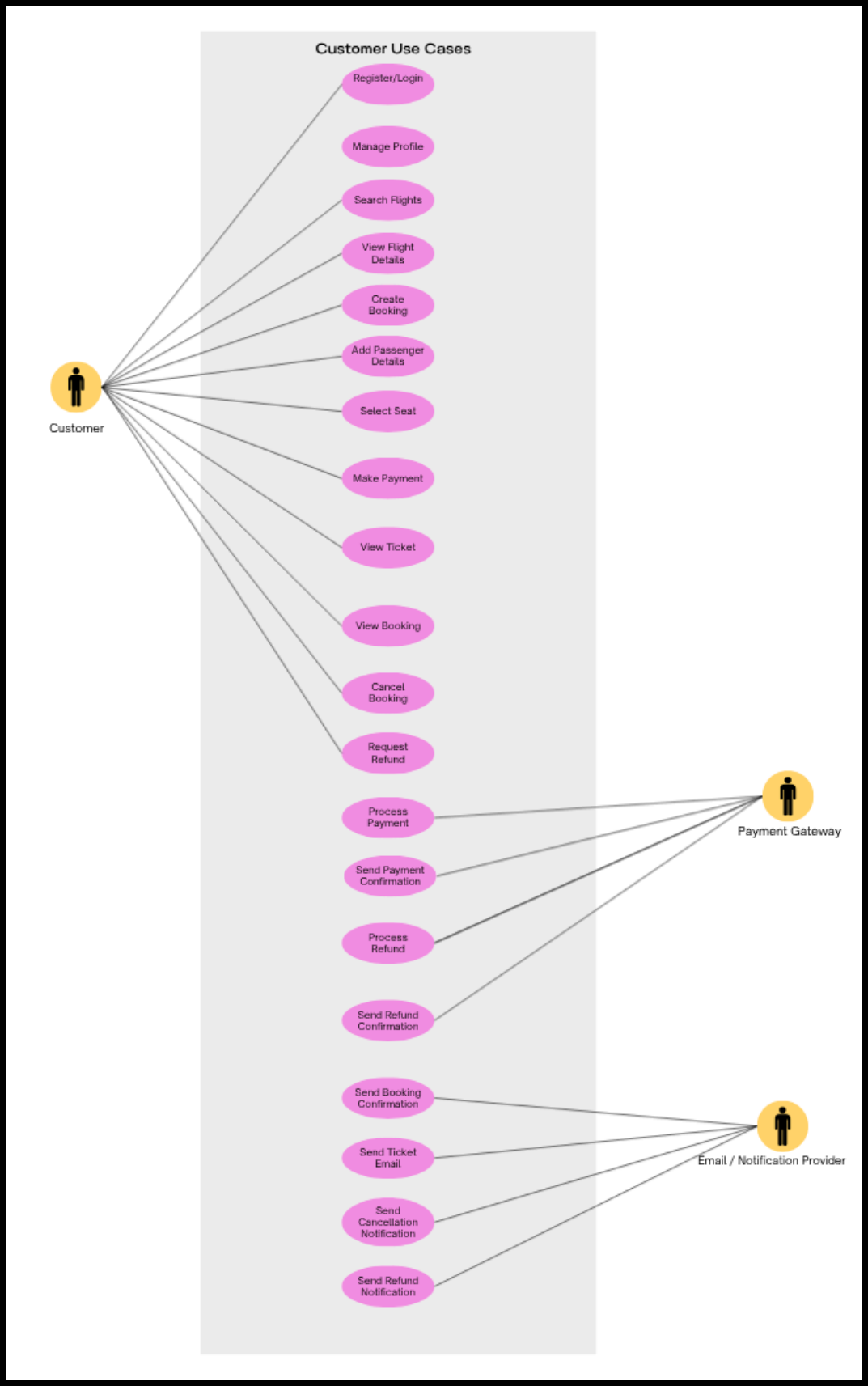
3. *Modular Monolith*

1. Identity and Access Management Module
2. Administration Module
3. Catalog Module
4. Reservation Module
5. Billing Module
6. Ticketing Module
7. Notification Module
8. Reporting Module

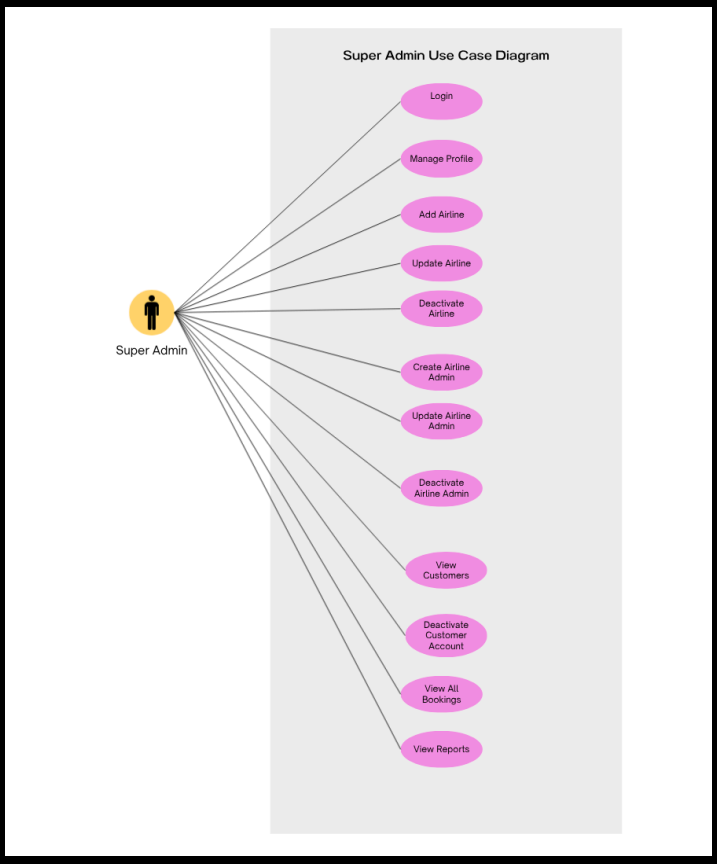
---

4. *USE CASES Diagram :*

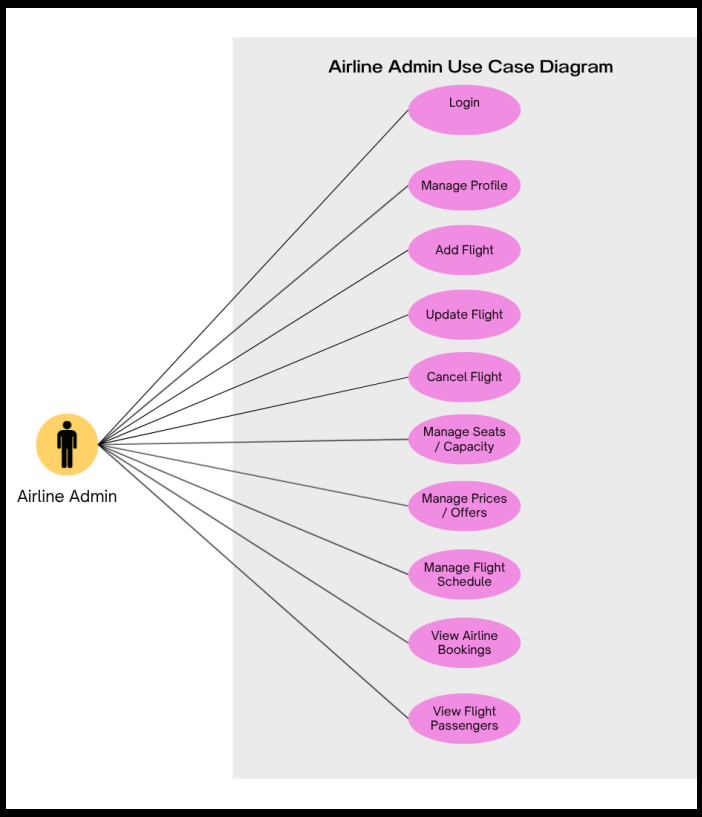
customer



2. Super Admin



3. Airline Admin



5. ***Functional Requirements***

Each actor in the system has its own functional requirements based on the actions they are allowed to perform.

- **Customer Functional Requirements**

1. Customers can register and log in to their accounts.
2. Customers can manage their personal profile information.
3. Customers can search for available flights.
4. Customers can view flight details, including route, schedule, price, and availability.
5. Customers can create a booking for a selected flight.
6. Customers can add passenger details during the booking process.
7. Customers can select available seats for the selected flight.
8. Customers can make payments through a payment gateway.
9. Customers can view their tickets after successful booking confirmation.
10. Customers can view their bookings.
11. Customers can cancel a booking.
12. Customers can request a refund for eligible paid bookings.

- **Payment Gateway Functional Requirements**

1. The system integrates with a payment gateway to process customer payments.
2. The payment gateway returns the payment result to the system.
3. The system updates the payment status based on the payment gateway response.
4. The system supports refund processing through the payment gateway for eligible bookings.
5. The payment gateway returns the refund result to the system.
6. The system updates the refund status based on the refund result.

- **Notification Functional Requirements**

1. The system sends a booking confirmation notification after a booking is successfully confirmed.
2. The system sends the ticket details to the customer by email after successful payment and ticket generation.
3. The system sends a cancellation notification after a booking is cancelled.
4. The system sends a refund notification after a refund is processed.
5. The system sends email notifications to affected customers when any flight information is updated, including schedule changes or flight cancellation.

- **Airline Admin Functional Requirements**

6. Airline admins can log in to their accounts.
7. Airline admins can manage their profiles.
8. Airline admins can add flights for their assigned airline.
9. Airline admins can update flight information for their assigned airline.
10. Airline admins can cancel flights for their assigned airline.
11. Airline admins can manage flight schedules.
12. Airline admins can manage seats and flight capacity.
13. Airline admins can manage flight prices and offers.
14. Airline admins can view bookings related to their assigned airline only.
15. Airline admins can view passengers for flights belonging to their assigned airline only.

- **Super Admin Functional Requirements**

1. The super admin can log in to the system.
2. The super admin can manage their profile.
3. The super admin can add airlines.
4. The super admin can update airline information.
5. The super admin can deactivate airlines.
6. The super admin can create airline admin accounts.
7. The super admin can update airline admin accounts.
8. The super admin can deactivate airline admin accounts.
9. The super admin can view customer accounts.
10. The super admin can deactivate customer accounts.
11. The super admin can view all bookings across the system.
12. The super admin can view system reports.

---

6. ***Non-Functional Requirements***

- **Security Requirements**

7. The system secures all protected APIs using authentication and authorization.
8. The system applies role-based access control to separate customer, airline admin, and super admin permissions.
9. Airline admins can access and manage data related only to their assigned airline.
10. Customers cannot access bookings that do not belong to them.
11. The system protects payment-related records, such as payment tokens, transaction references, and payment statuses, and does not store sensitive card information.
12. The system stores user passwords securely using password hashing.
13. The system restricts super admin operations to authorized super admin accounts only.
14. The system validates authentication tokens before allowing access to protected APIs.
15. The system verifies payment gateway responses before updating payment, booking, or refund statuses.

- **Data Consistency and Integrity Requirements**

1. The system maintains seat availability consistency during concurrent booking requests.
2. The system updates booking, payment, ticket, and refund statuses consistently.
3. The system creates a booking with a pending payment status before payment confirmation.
4. The system confirms a booking only after successful payment.
5. The system generates a ticket only after booking confirmation.
6. The system does not delete historical booking, payment, ticket, or refund records.

- **Performance Requirements**

1. The system returns flight search results within an acceptable response time.
2. The system supports filtering flights by source, destination, date, passenger count, and availability.
3. The system handles multiple users searching and booking flights at the same time.

4. The system uses database indexing where needed to improve search and booking performance.

- **Reliability Requirements**

1. The system handles external service failures, such as payment gateway or email provider failures, without losing data or corrupting booking, payment, or notification statuses.
2. The system keeps bookings in a pending payment state if payment confirmation is delayed.
3. The system handles failed payments and failed refunds safely.
4. The system logs important operations such as booking creation, payment results, cancellations, and refund results.

- **Availability Requirements**

1. The system remains available for customers to search flights and view bookings.
2. The system remains available for admins to manage flights and view bookings when authenticated.
3. The system handles external service downtime, such as payment gateway or email provider failure, without crashing the whole application.

- **Maintainability Requirements**

1. The system is organized using a modular monolith structure.
2. The system separates business modules such as authentication, flight management, booking, payment, ticketing, notification, and admin management.
3. The system keeps payment logic separated from booking logic.
4. The system keeps notification logic separated from payment and booking logic.
5. The system uses clear service, repository, controller, entity, and DTO layers.

- **Scalability Requirements**

6. The system design allows future scaling of search, booking, payment, and notification features.
7. The system supports integration with different payment providers without major changes to the core booking and payment logic.
8. The system supports extending notification channels, such as email or future SMS notifications, without affecting the booking and payment

modules.
9. The system supports adding more airlines and airline admins without changing the core design.

- **Usability Requirements**

1. The system provides clear booking, payment, cancellation, and refund statuses to the customer.
2. The system provides clear error messages when payment fails, booking fails, or a selected seat is unavailable.
3. The system allows admins to manage flights and bookings through clear and organized operations.

- **Audit and Logging Requirements**

1. The system keeps logs for sensitive admin actions such as creating airline admins, deactivating airlines, and deactivating customer accounts.
2. The system keeps logs for payment and refund results.
3. The system keeps enough historical data to support reports and troubleshooting.

- **Auth Business Rules**

1. Customer registration is public.
2. Login is public for all user roles.
3. Protected Auth endpoints require a valid JWT token.
4. The JWT token must be sent in the Authorization header.
5. The system uses role-based access control.
6. User roles are CUSTOMER, AIRLINE_ADMIN, and SUPER_ADMIN.
7. Airline Admin accounts are created only by the Super Admin.
8. Passwords must never be stored as plain text.
9. Deactivated users cannot access protected endpoints.

---

7. ***Business Rules***

- **User and Role Rules**

8. Airline admins cannot register directly from the public registration page.
9. Airline admin accounts are created only by a super admin.
10. Each airline admin must be assigned to one airline.
11. Airline admins can manage data related only to their assigned airline.
12. Customers can access only their own bookings and tickets.
13. Super admin operations are restricted to authorized super admin accounts only.

- **Airline Rules**

1. Airlines are deactivated instead of being permanently deleted.
2. An inactive airline cannot have new flights added.
3. Old flights, bookings, payments, and tickets related to a deactivated airline remain stored in the system.

- **Flight Management Rules**

1. Airline admins cannot add, update, or cancel flights belonging to other airlines.
2. Each flight must include route, schedule, price, seat capacity, and availability information.
3. When flight information is updated, affected customers must be notified by email.
4. When a flight is cancelled, customers with bookings on that flight must be notified by email.

- **Search and Flight Selection Rules**

1. Customers can create a booking only for an available flight.
2. Customers can select seats only from the available seats of the selected flight.
3. Flight search results must reflect current flight availability.

- **Seat Availability Rules**

1. Each seat on a flight can be assigned to only one passenger.
2. Seat availability must be checked before creating a booking.
3. Seat availability must remain consistent when multiple customers try to book at the same time.
4. Once a booking is confirmed, the selected seat becomes booked.
5. If a booking is cancelled before confirmation, the selected seat can become available again.

- **Booking Rules**

1. A booking is created with a PENDING_PAYMENT status before payment confirmation.
2. A booking becomes CONFIRMED only after successful payment.
3. A booking becomes FAILED if the payment fails or the booking process cannot be completed.
4. A booking becomes EXPIRED if the customer does not complete payment within the allowed time.
5. A booking becomes CANCELLED if the customer cancels an eligible booking or if the booking is cancelled by the system/admin.
6. A ticket cannot be generated for a booking that is not confirmed.
7. Cancelled bookings remain stored in the system for history and reports.

- **Passenger Rules**

1. A booking can contain one or more passenger records.
2. Passenger details must be added during the booking process.
3. A passenger does not have to be a registered user.
4. The customer who creates the booking is responsible for entering correct passenger details.

- **Payment Rules**

1. Payment status is updated based on the payment gateway response.
2. A booking remains PENDING_PAYMENT if payment confirmation is delayed.
3. A booking is confirmed only when the payment result is successful.
4. If payment fails, the booking is not confirmed and no ticket is generated.
5. Payment gateway responses must be verified before updating payment, booking, or refund statuses.

6. If a customer cancels a paid booking and requests a refund, the system updates the refund status after receiving the refund result from the payment gateway.
7. The system stores only necessary payment references, transaction IDs, payment tokens, payment amounts, and payment statuses.
8. The system does not store sensitive card information.

- **Ticket Rules**

1. A ticket is generated only after booking confirmation.
2. A ticket is linked to a confirmed booking.
3. A customer can view the ticket only after the related booking is confirmed.
4. Ticket details are sent to the customer by email after successful payment and ticket generation.
5. Ticket records remain stored in the system and are not deleted.

- **Cancellation Rules**

1. If a booking is cancelled before payment, no refund is required.
2. If a confirmed paid booking is cancelled, the system checks refund eligibility.
3. A cancelled booking keeps its historical record in the system.
4. The system sends a cancellation email to the customer after booking cancellation.

- **Refund Rules**

1. Customers can request a refund only for eligible paid bookings.
2. A refund cannot be requested for an unpaid booking.
3. Refund processing is handled through the payment gateway.
4. The refund status is updated based on the payment gateway response.
5. Failed refunds are recorded and handled safely.
6. The system sends a refund notification after the refund result is processed.

- **Notification Rules**

1. The system sends a booking confirmation notification after a booking is confirmed.
2. The system sends ticket details to the customer by email after ticket generation.
3. The system sends a cancellation notification after booking cancellation.
4. The system sends a refund notification after refund processing.
5. The system sends email notifications to affected customers when any flight information is updated, including schedule changes or flight cancellation.
6. Notification failure must not cancel the booking or payment operation.

- **Data History and Audit Rules**

1. Historical booking, payment, ticket, and refund records are not deleted.
2. Sensitive admin actions must be logged.
3. Payment and refund results must be logged.
4. Booking creation, cancellation, payment results, and refund results are kept for reports and troubleshooting.
5. Deactivation is preferred over deletion for airlines, airline admins, and customer accounts to preserve historical data.

- **Reporting Rules**

1. Super admin reports can include all airlines and all bookings.
2. Airline admins can view bookings and passengers related only to their assigned airline.
3. Reports can use historical booking, payment, refund, airline, and customer data.

---

8. *Enumeration Types (Enums)*

This section defines the fixed status values and categories used across the Flight Booking System.

1. User Enums

User Role

- CUSTOMER
- AIRLINE_ADMIN
- SUPER_ADMIN

User Status

- ACTIVE
- BLOCKED
- DEACTIVATED

---

2. Airline Enums

Airline Status

- ACTIVE
- INACTIVE

---

3. Flight Enums

Flight Status

- SCHEDULED
- DELAYED
- CANCELLED
- COMPLETED

Cabin Class

- ECONOMY
- BUSINESS
- FIRST_CLASS

---

4. Booking Enums

Booking Status

- PENDING_PAYMENT
- CONFIRMED
- CANCELLED
- EXPIRED
- FAILED

---

5. Payment and Refund Enums

Payment Status

- PENDING
- SUCCESS
- FAILED

Refund Status

- NOT_REQUESTED
- PENDING
- REFUNDED
- FAILED

---

6. Ticket Enums

Ticket Status

- ISSUED
- CANCELLED

---

7. Notification Enums

Notification Type

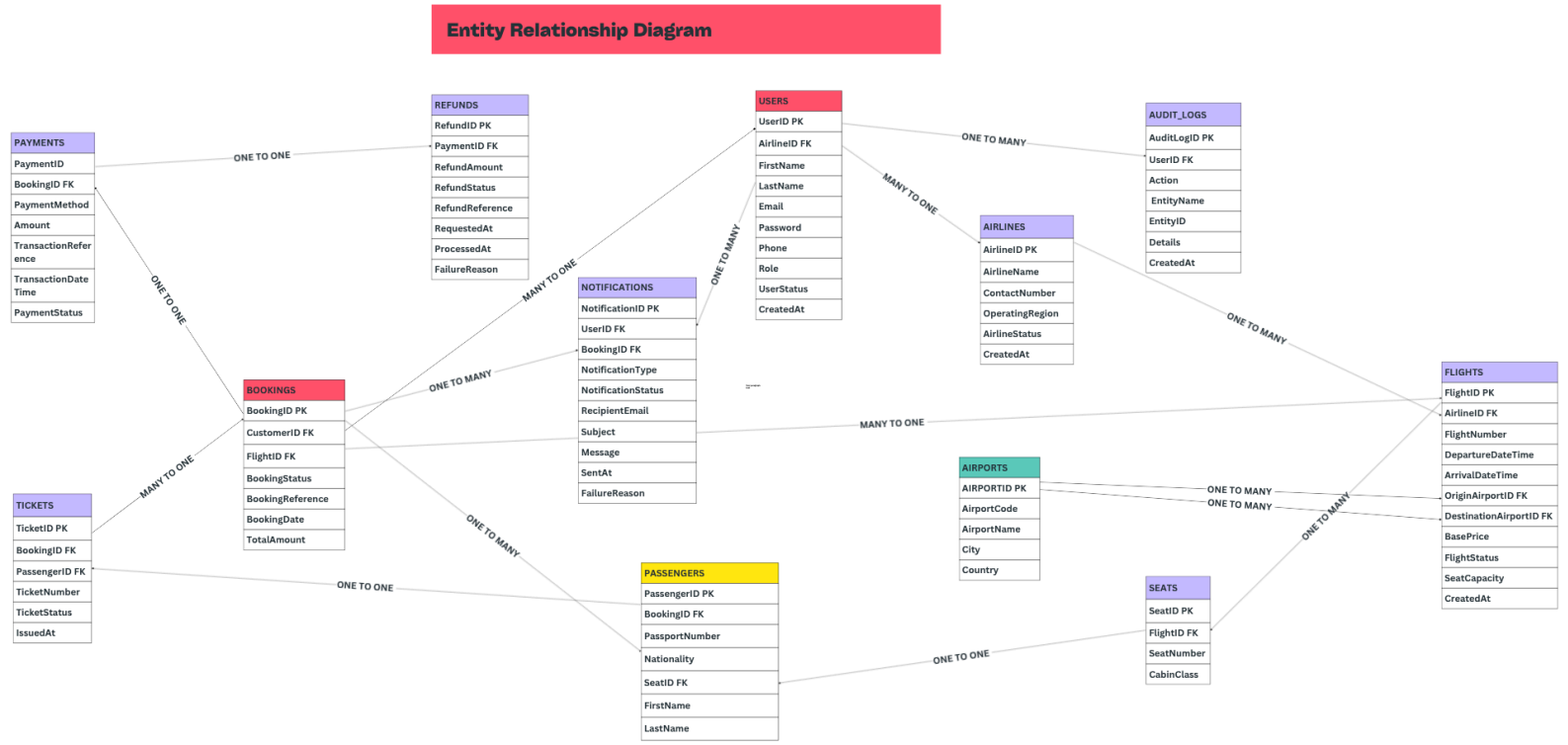
- BOOKING_CONFIRMATION
- TICKET_EMAIL
- CANCELLATION_EMAIL
- REFUND_EMAIL
- FLIGHT_UPDATE_EMAIL

Notification Status

- PENDING
- SENT
- FAILED

---

9. ERD


Entity Relationship Diagram
PAYMENTS
PaymentID
BookingID FK
PaymentMethod
Amount
TransactionReference
TransactionDateTime
PaymentStatus
REFUNDS
RefundID PK
PaymentID FK
RefundAmount
RefundStatus
RefundReference
RequestedAt
ProcessedAt
FailureReason
USERS
UserID PK
AirlineID FK
FirstName
LastName
Email
Password
Phone
Role
UserStatus
CreatedAt
AUDIT_LOGS
AuditLogID PK
UserID FK
Action
EntityName
EntityID
Details
CreatedAt
AIRLINES
AirlineID PK
AirlineName
ContactNumber
OperatingRegion
AirlineStatus
CreatedAt
NOTIFICATIONS
NotificationID PK
UserID FK
BookingID FK
NotificationType
NotificationStatus
RecipientEmail
Subject
Message
SentAt
FailureReason
BOOKINGS
BookingID PK
CustomerID FK
FlightID FK
BookingStatus
BookingReference
BookingDate
TotalAmount
FLIGHTS
FlightID PK
AirlineID FK
FlightNumber
DepartureDateTime
ArrivalDateTime
OriginAirportID FK
DestinationAirportID FK
BasePrice
FlightStatus
SeatCapacity
CreatedAt
AIRPORTS
AIRPORTID PK
AirportCode
AirportName
City
Country
TICKETS
TicketID PK
BookingID FK
PassengerID FK
TicketNumber
TicketStatus
IssuedAt
PASSENGERS
PassengerID PK
BookingID FK
PassportNumber
Nationality
SeatID FK
FirstName
LastName
SEATS
SeatID PK
FlightID FK
SeatNumber
CabinClass
ONE TO ONE
ONE TO MANY
MANY TO ONE
MANY TO ONE
ONE TO MANY
ONE TO MANY
ONE TO MANY
MANY TO ONE
ONE TO MANY
ONE TO MANY
ONE TO MANY
ONE TO ONE
ONE TO ONE
ONE TO MANY

10. ***Relationships***

1. AIRLINES to USERS = ZERO TO MANY

   USERS to AIRLINES = ZERO TO ONE

2. AIRLINES to FLIGHTS = ZERO TO MANY

   FLIGHTS to AIRLINES = MANY TO ONE

3. AIRPORTS to FLIGHTS = ZERO TO MANY

   FLIGHTS to AIRPORTS = MANY TO ONE

   Relation: Origin Airport

4. AIRPORTS to FLIGHTS = ZERO TO MANY

   FLIGHTS to AIRPORTS = MANY TO ONE

   Relation: Destination Airport

5. USERS to BOOKINGS = ZERO TO MANY

   BOOKINGS to USERS = MANY TO ONE

6. FLIGHTS to BOOKINGS = ZERO TO MANY

   BOOKINGS to FLIGHTS = MANY TO ONE

7. BOOKINGS to PASSENGERS = ONE TO MANY

   PASSENGERS to BOOKINGS = MANY TO ONE

8. FLIGHTS to SEATS = ONE TO MANY

   SEATS to FLIGHTS = MANY TO ONE

9. SEATS to PASSENGERS = ZERO TO ONE

   PASSENGERS to SEATS = ONE TO ONE

10. BOOKINGS to PAYMENTS = ZERO TO ONE

    PAYMENTS to BOOKINGS = ONE TO ONE

11. PAYMENTS to REFUNDS = ZERO TO ONE

    REFUNDS to PAYMENTS = ONE TO ONE

12. BOOKINGS to TICKETS = ZERO TO MANY

    TICKETS to BOOKINGS = MANY TO ONE

13. PASSENGERS to TICKETS = ZERO TO ONE

    TICKETS to PASSENGERS = ONE TO ONE

14. USERS to NOTIFICATIONS = ZERO TO MANY

    NOTIFICATIONS to USERS = MANY TO ONE

15. BOOKINGS to NOTIFICATIONS = ZERO TO MANY

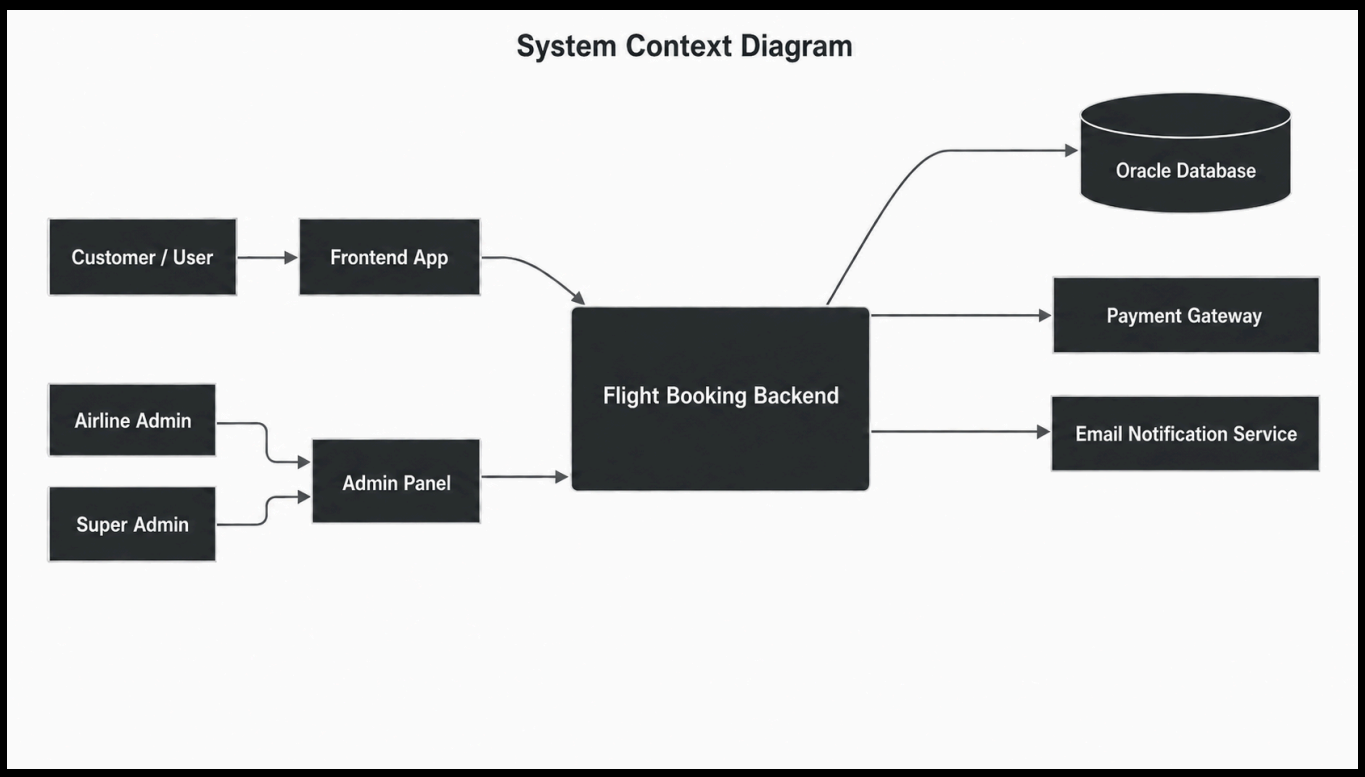
    NOTIFICATIONS to BOOKINGS = ZERO TO ONE

16. USERS to AUDIT_LOGS = ZERO TO MANY

    AUDIT_LOGS to USERS = MANY TO ONE

---

11. *System Context Diagram*



12. ***API Contract***

https://github.com/KholoudShaarawii/flight-booking-system/blob/main/docs/Full%20API%20Contract.pdf